Slurm Tutorial

A hands-on tutorial for our Slurm cluster. Work through the Tutorial Code Snippets section command by command — every snippet has a matching runnable file in `examples/` so you can submit it for real.

> **Conventions:** angle-bracket placeholders like `<your-username>` need to be replaced with your real values.

What is Slurm

Slurm (Simple Linux Utility for Resource Management) is a *workload manager / job scheduler* for HPC clusters. Instead of everyone SSHing onto machines and fighting over CPUs and GPUs, you describe the resources your job needs and Slurm queues it, finds a node that fits, runs it, and cleans up afterwards. This keeps the cluster fairly shared and fully utilised.

Key concepts:

- **Login node** — where you land when you SSH in. Use it to edit files and submit jobs. **Do not run heavy work here.**
- **Compute nodes** — the machines that actually run your jobs.
- **`slurmctld`** — the controller daemon that schedules everything.
- **Partition** — a named queue of nodes (e.g. `cpu`, `gpu`, `short`). You pick one per job.
- **Job vs. step** — a *job* is one submission (`sbatch`/`srun`); a job can run multiple *steps* (each `srun` inside it).
- **Resource request** — CPUs, memory, time, and GPUs you ask for. Slurm grants exactly that, so request what you actually use.

The job lifecycle

```
submit  →  PENDING (in queue)  →  RUNNING  →  COMPLETED / FAILED
```

You don't have to stay connected for batch jobs — submit and log off, the job keeps running. `squeue` shows where your job currently sits in this cycle.

Basic Commands

Command	What it does
`sinfo`	Show partitions and node states
`squeue`	Show queued/running jobs (`squeue --me` for just yours)
`sbatch`	Submit a batch job script
`srun`	Run a command/step (interactively or inside a job)

Command	What it does
`salloc`	Grab an interactive allocation (a shell on a node)
`scancel`	Cancel a job (`scancel <jobid>`)
`sacct`	Accounting/history for finished jobs
`scontrol`	Inspect/modify jobs, nodes, partitions

Cluster Information

```
# Partitions, availability, time limits, node counts
sinfo

# One line per node, long format (state, CPUs, memory)
sinfo -N -l

# Everything about one node, including Gres (GPUs)
scontrol show node <nodename>

# Partition limits (max time, default mem, allowed accounts)
scontrol show partition <partition>
```

Reading `sinfo`: the `STATE` column tells you what's usable — `idle` (free), `mix` (partly used), `alloc` (full), `down`/`drain` (unavailable).

Tutorial Code Snippets

First Connection to the Slurm Cluster

1. **Create an SSH key** (skip if you already have one):

   ```
   ssh-keygen -t ed25519 -C "<your-uni-username>@uni-rostock"
   ```

2. **Add it to your agent** so you don't retype the passphrase:

   ```
   eval "$(ssh-agent -s)"
   ssh-add ~/.ssh/id_ed25519
   ```

3. **Copy the public key to the server:**

   ```
   ssh-copy-id <your-uni-username>@sl-li.informatik.uni-rostock.de
   ```

4. **Add a host block** to `~/.ssh/config` so you can just type `ssh slurm`. `ForwardAgent yes` lets the login node reuse your local key (e.g. to clone from GitHub) without copying the private key onto the cluster.

```
Host slurm
    HostName sl-li.informatik.uni-rostock.de
    User <your-uni-username>
    ForwardAgent yes
    # ProxyJump <gateway>   # uncomment only if you must hop through a
gateway
```

5. **Connect:**

```
ssh slurm
```

Overview over the cluster

Once you're on the login node, get the lay of the land:

```
sinfo                # which partitions and nodes exist, and their state
squeue --me          # your jobs
squeue               # everyone's jobs (how busy is it?)
sinfo -N -l          # detailed per-node view (CPUs, memory, state)
```

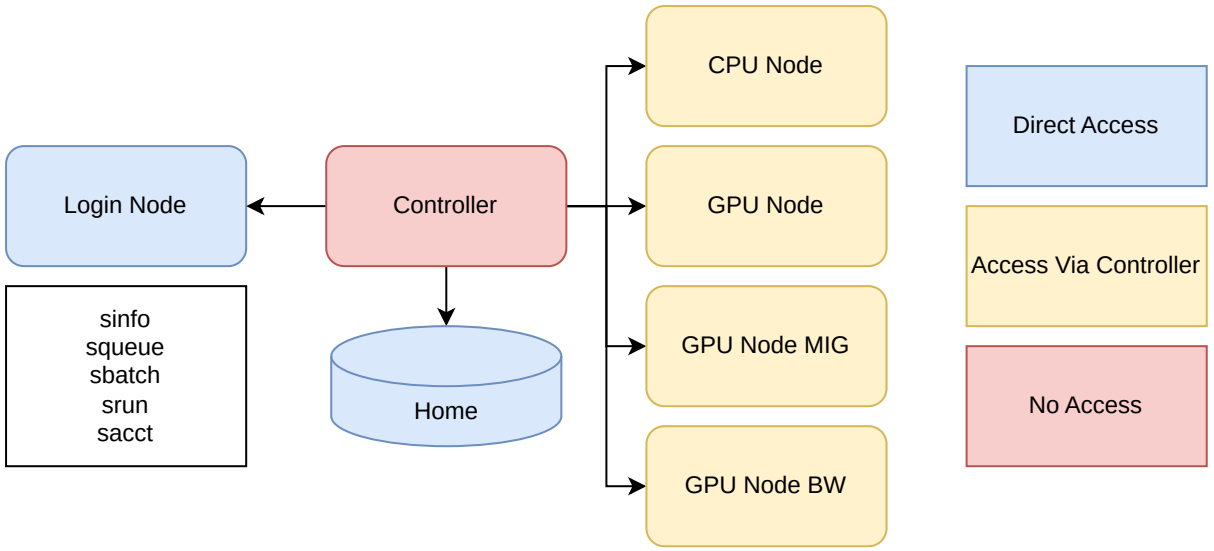
Example `sinfo` output:

```
PARTITION    AVAIL  TIMELIMIT  NODES  STATE NODELIST
compute-node    up 1-00:00:00      4   idle node[001-004]
gpu-node-mig    up 1-00:00:00      1   idle node201
login-node      up 1-00:00:00      1   idle sl-li
gpu-node        up 1-00:00:00      2   idle node[101-102]
gpu-node-bw     up 1-00:00:00      2   idle node[121-122]
```

The `TIMELIMIT` column (`1-00:00:00` = 1 day) is the **max** wall-clock a job may request on that partition — ask for more and it's rejected.

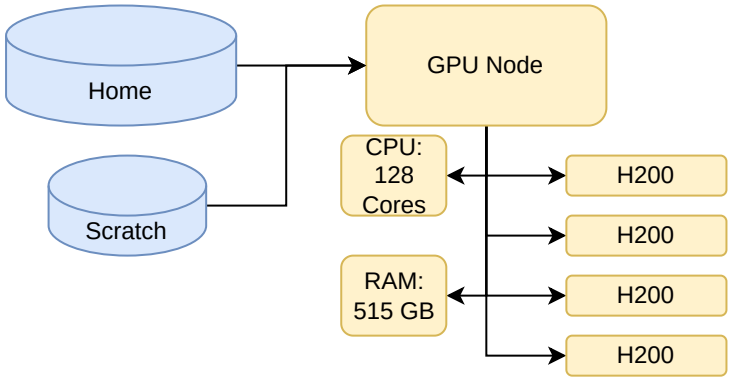
Our partitions: **`compute-node`** (CPU jobs), **`gpu-node-mig`** (small GPU slices via MIG — great for tutorials/dev), and **`gpu-node`** (full GPUs for heavier work). Examples below use `compute-node` and `gpu-node-mig`.

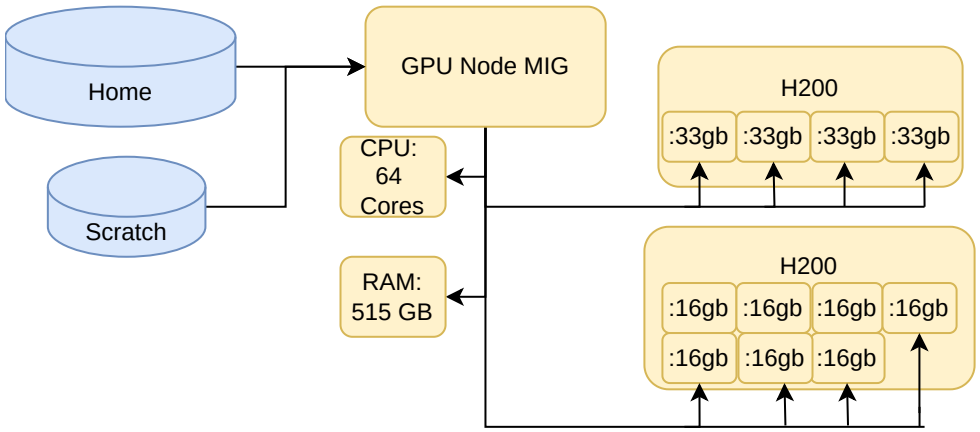
The hardware



- **GPU nodes** (`gpu-node`) — each node has **4× NVIDIA H200** GPUs. Use these for full-GPU training and heavier work.
- **GPU nodes** (`gpu-node-bw`) — each node has **NVIDIA B6000 RTX** GPUs.
- **MIG node** (`gpu-node-mig`, `node201`) — **2× H200** sliced with MIG (Multi-Instance GPU) into smaller, independent GPUs:
  - one H200 → **4 slices of ~33 GB** each,
  - one H200 → **7 slices of ~16 GB** each.

  MIG slices are ideal for tutorials, development, and inference where a whole H200 would be overkill — you get a guaranteed slice without waiting for a full GPU.
- **Compute nodes** (`compute-node`) — CPU-only, for preprocessing and non-GPU jobs.





Requesting MIG slices on `gpu-node-mig`:

- **Any** slice — just `--gres=gpu:1`; Slurm hands you whichever is free.
- A **specific size** — name the MIG type: `--gres=gpu:1g.33gb:1` (≈33 GB, 4 available) or `--gres=gpu:1g.16gb:1` (≈16 GB, 7 available). The type strings come from the node's GRES config:

```
scontrol show node node201 | grep -i Gres
# Gres=gpu:1g.33gb:4,gpu:1g.16gb:7
```

Ready-made scripts: `examples/gpu_mig_33gb.sbatch` and `examples/gpu_mig_16gb.sbatch`.

srun vs. sbatch vs. salloc

Three ways to get work onto a node — pick by how interactive it is:

- **`srun`** — run a command now; blocks until it finishes. Good for quick tests and interactive shells.
- **`sbatch`** — submit a script that runs when a slot frees up. The workhorse for real jobs; you can log off while it runs.
- **`salloc`** — grab an interactive allocation (a shell on a node) to poke around by hand.

SRUN — run something now

`srun` runs a command on a compute node and blocks until it finishes. Great for quick tests and interactive work.

```
# Run our Monte-Carlo pi estimate on a compute node, 1 CPU:
srun --partition=compute-node --cpus-per-task=1 --time=00:05:00 \
     python examples/pi_estimate.py 5000000

# Get an interactive shell on a compute node (great for debugging):
srun --partition=compute-node --pty bash
```

Source: `examples/pi_estimate.py`.

SBATCH — submit a batch job

For real work you write a *batch script*: a normal shell script whose `#SBATCH` lines describe the resource request. You submit it and Slurm runs it whenever a slot is free — no need to stay connected.

Every batch script should specify at least: **partition**, **time limit**, **memory**, **CPUs**, and (if you need a GPU) **GRES**.

```
#!/bin/bash
#SBATCH --job-name=pi-demo
#SBATCH --partition=compute-node # which queue (see `sinfo`)
#SBATCH --cpus-per-task=1        # CPUs for your program
```

```
#SBATCH --mem=512M                # RAM per node
#SBATCH --time=00:05:00           # hard wall-clock limit (HH:MM:SS)
#SBATCH --output=logs/%x-%j.out   # %x=job name, %j=job id

echo "Running on host: $(hostname)"
srun python examples/pi_estimate.py 5000000
```

Submit and watch it:

```
mkdir -p logs                  # the --output path must exist
sbatch examples/srun_demo.sbatch
squeue --me                    # watch it queue and run
cat logs/pi-demo-*.out         # read the output when done
```

GPU jobs add a `--gres` line to request a GPU:

```
#SBATCH --partition=gpu-node-mig
#SBATCH --gres=gpu:1           # 1 GPU (a MIG slice on this partition)
```

Full files: `examples/srun_demo.sbatch` (CPU) and `examples/gpu.sbatch` + `examples/gpu_check.py` (GPU).

Logs & output (important!)

A batch job runs detached — you're not watching the terminal — so **its log file is your only window into what happened.** Always set `--output`, and check it.

- `#SBATCH --output=logs/%x-%j.out` captures **stdout**. The `%x` (job name) and `%j` (job id) placeholders keep files unique so jobs don't overwrite each other. Without `--output`, Slurm dumps everything into `slurm-<jobid>.out` in your current directory — easy to lose track of.
- `#SBATCH --error=logs/%x-%j.err` sends **stderr** to a separate file. Handy for spotting failures fast; omit it and stderr is merged into the `.out`.
- The `logs/` directory **must exist first** — `mkdir -p logs` before submitting.
- **Watch a running job live:**

  ```
  tail -f logs/pi-demo-12345.out
  ```

- In Python, `print(..., flush=True)` (or `python -u`) so progress shows up in the log immediately instead of being buffered until the job ends — see `examples/checkpoint.py`.

When a job fails, the log is the **first** place to look (`sacct -j <jobid>` tells you the exit state; the log tells you *why*).

Development Workflow (git-based)

Don't edit code directly on the cluster. Develop and test on your laptop, push to git, and pull on the cluster — the cluster just *runs* what's in git. This keeps your laptop and the cluster in sync and your history clean.

```
laptop:  edit + test  →  git commit  →  git push
                                            |
cluster:                          git pull  →  sbatch  →  read logs
```

One-time setup on the cluster (uses agent forwarding from your SSH config, so your laptop's key authenticates to GitHub — nothing private is copied over):

```
ssh slurm
git clone git@github.com:<you>/<repo>.git
cd <repo>
uv sync                       # reproduce the Python env from pyproject.toml
```

Each iteration:

```
# 1. On your laptop: make changes, test quickly, then publish
git add -A && git commit -m "tweak training step"
git push

# 2. On the cluster: pull and submit
ssh slurm
cd <repo>
git pull
sbatch examples/srun_demo.sbatch
squeue --me                   # watch it
cat logs/*.out                # inspect results, then repeat
```

> **Tip:** keep `logs/` and any large outputs out of git (add them to `.gitignore`) — commit code, not run artifacts.

Advanced

Job Dependencies (build a pipeline)

You often want job B to run only after job A succeeds — e.g. preprocess, then train. Capture A's job id with `--parsable` and pass it to B's `--dependency`:

```
mkdir -p logs
# Submit step A and grab its job id
jid=$(sbatch --parsable examples/step_a.sbatch)
```

```
echo "step A is job $jid"

# Submit step B; it waits until A finishes successfully (afterok)
sbatch --dependency=afterok:$jid examples/step_b.sbatch

squeue --me   # step B shows state (Dependency) until A is done
```

Common dependency types: `afterok` (A succeeded), `afterany` (A finished, any result), `afternotok` (A failed). Files: `examples/step_a.sbatch`, `examples/step_b.sbatch`, `examples/primes.py`.

Send Mail

Let Slurm email you when a job changes state:

```
#SBATCH --mail-type=BEGIN,END,FAIL   # or ALL
#SBATCH --mail-user=<you>@example.com
```

Run Jupyter Lab on a compute node

The idea: Jupyter runs on a **compute node**, and you reach it from your laptop's browser through an SSH tunnel that hops via the login node. Step by step:

1. **SSH into the cluster:**

   ```
   ssh slurm
   ```

2. **Make sure Jupyter is available** in your environment. With this repo's uv project:

   ```
   cd slurmtutorial
   uv sync                  # installs jupyterlab (+ torch) from
   pyproject.toml
   ```

   (Or `module load`/activate whatever environment you normally use.)

3. **Launch Jupyter on a compute node.** This grabs a GPU slice and starts the server, binding to all interfaces so the tunnel can reach it:

   ```
   srun --partition=gpu-node-mig --gres=gpu:1 --time=04:00:00 --pty \
        uv run jupyter lab --no-browser --ip=0.0.0.0 --port=8888
   ```

   No GPU needed? Use `--partition=compute-node` and drop `--gres=gpu:1`.

4. **Note two things from the output:**

   - the **node name** it landed on — run `hostname` in another shell on that allocation, or read it from the prompt (e.g. `node201`);
   - the **token URL** it prints, like `http://127.0.0.1:8888/lab?token=abc123...`.

5. **Open the tunnel from your laptop** (new terminal, *not* on the cluster). This forwards local port 8888 to the compute node, hopping through the login node:

   ```
   ssh -N -L 8888:<node>:8888 slurm     # e.g. 8888:node201:8888
   ```

   Leave this running.

6. **Open Jupyter** in your browser: paste the token URL from step 4, but use the local address — `http://localhost:8888/lab?token=abc123...`.

7. **When you're done:** `Ctrl-C` the `srun` session on the cluster (this ends the job and frees the GPU), then `Ctrl-C` the tunnel on your laptop.

> **Tip:** if port 8888 is already taken, pick another (e.g. `8889`) and use it consistently in steps 3, 5, and 6.

Requeue after time limit (with checkpointing)

For jobs longer than the partition's time limit, checkpoint progress and have Slurm requeue the job to resume. The script asks Slurm to send `SIGUSR1` shortly before the limit (`--signal=B:USR1@30`), traps it, saves state, and requeues:

```
mkdir -p logs
sbatch examples/requeue.sbatch
```

The Python side traps the signal, writes a checkpoint, and exits cleanly so the requeued run can pick up where it stopped. Files: `examples/requeue.sbatch`, `examples/checkpoint.py`.

Being a Good Cluster Citizen

The cluster is shared — a few habits keep it pleasant for everyone:

- **Never run heavy work on the login node.** Use `srun`/`sbatch` to push it to a compute node.
- **Request realistic time and memory.** Over-asking leaves resources idle and makes you wait longer in the queue.
- **Clean up.** `scancel` jobs you no longer need.
- **Pick the right partition** — `compute-node` for CPU work, `gpu-node-mig` for small/dev GPU jobs, `gpu-node` / `gpu-node-bw` for heavier GPU runs.